



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

DE52 Software Engineering

25 mars 2025 – Yazan MUALLA

Unified Modeling Language Diagrams

UML - Table of contents

- General Overview
- Structure and behavior diagrams
- Use-case diagrams
- Class diagrams
- Sequence diagrams
- State Machine diagrams

General Overview

- Unified Modeling Language (UML) is a standardized modeling language used in software engineering for visualizing, specifying, constructing, and documenting the structure and behavior of software systems
- It is adopted by the Object Management Group (OMG)
- UML is not a programming language but a set of graphical notations that help developers and stakeholders understand, design, and communicate system architecture

General Overview

■ Key Features of UML

- **Standardized:** It provides a universal approach to system modeling
- **Object-Oriented:** Built on object-oriented programming (OOP) principles
- **Graphical Representation:** Uses diagrams to represent system components and behaviors
- **Extensible:** Supports customization through UML Profiles and Stereotypes
- **Applicable Beyond Software:** Can be used for modeling business processes, workflows, and systems beyond software development

UML and Software Development Lifecycle

- UML is widely used in various phases of software development, including:
 - **Requirement Analysis:** Use Case diagrams help define system functionalities
 - **Design & Architecture:** Class, Component, and Deployment diagrams define system structure
 - **Implementation:** Developers use UML diagrams as blueprints for coding
 - **Testing & Maintenance:** State diagrams and Sequence diagrams help in test case design and debugging



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Structure and behavior diagrams

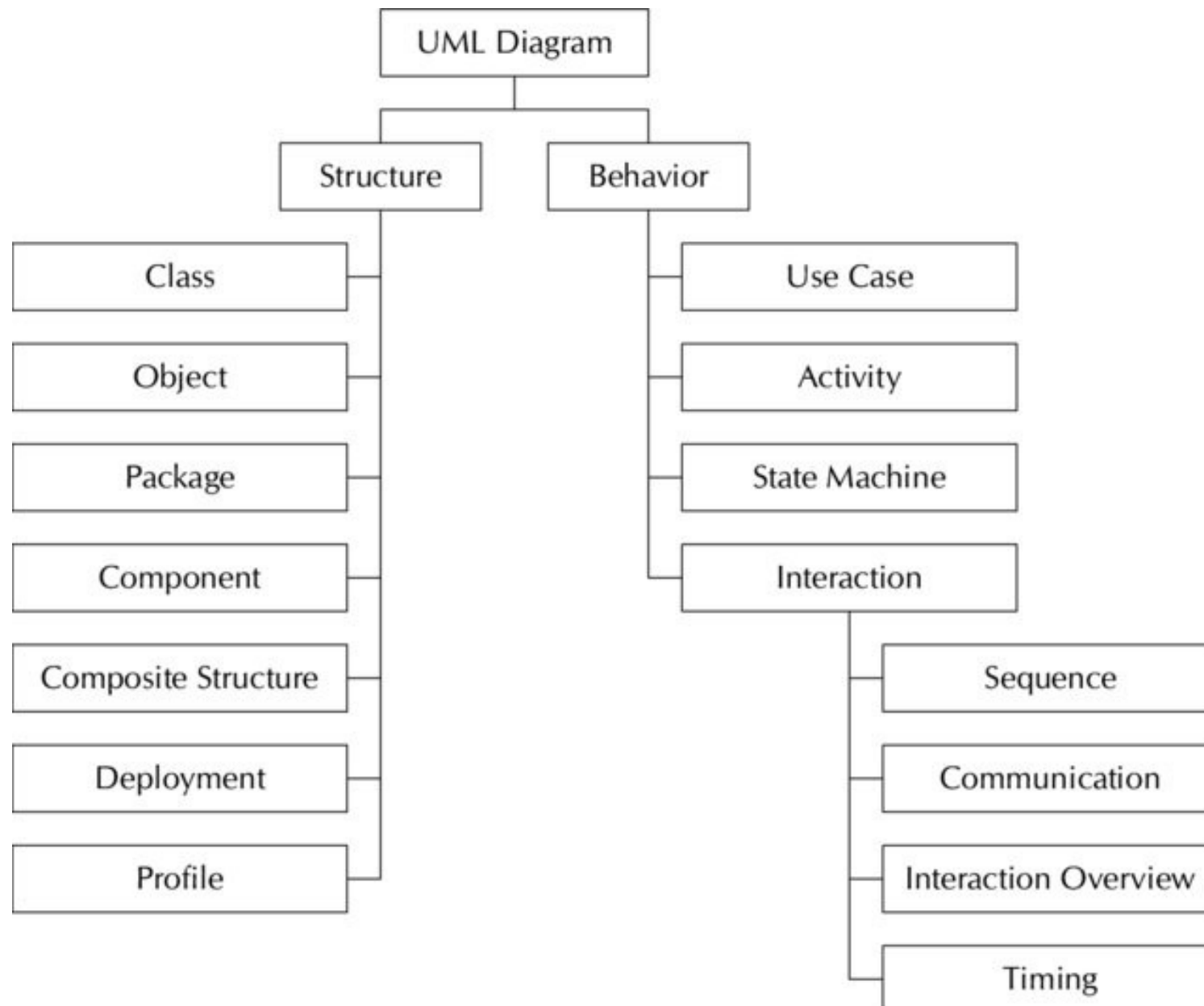
Structure diagrams

- Structure diagrams show the **static structure of the system** and its parts on different abstraction and implementation levels and how they are **related to each other**
- The elements in a structure diagram represent the meaningful concepts of a system, and may include abstract, real world and implementation concepts
- They help design the definition of **packages, class names, attributes, and method signatures** (but not method bodies)
- For example: UML class diagrams

Behavior diagrams

- Behavior diagrams show the **dynamic behavior of the objects in a system**, which can be described as a series of **changes** to the system **over time**
- They help design the logic, the behavior of the code or the method bodies
- For example: UML interaction diagrams (sequence diagrams or collaboration diagrams, a.k.a. communication diagrams)

UML 2.5 diagrams





UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

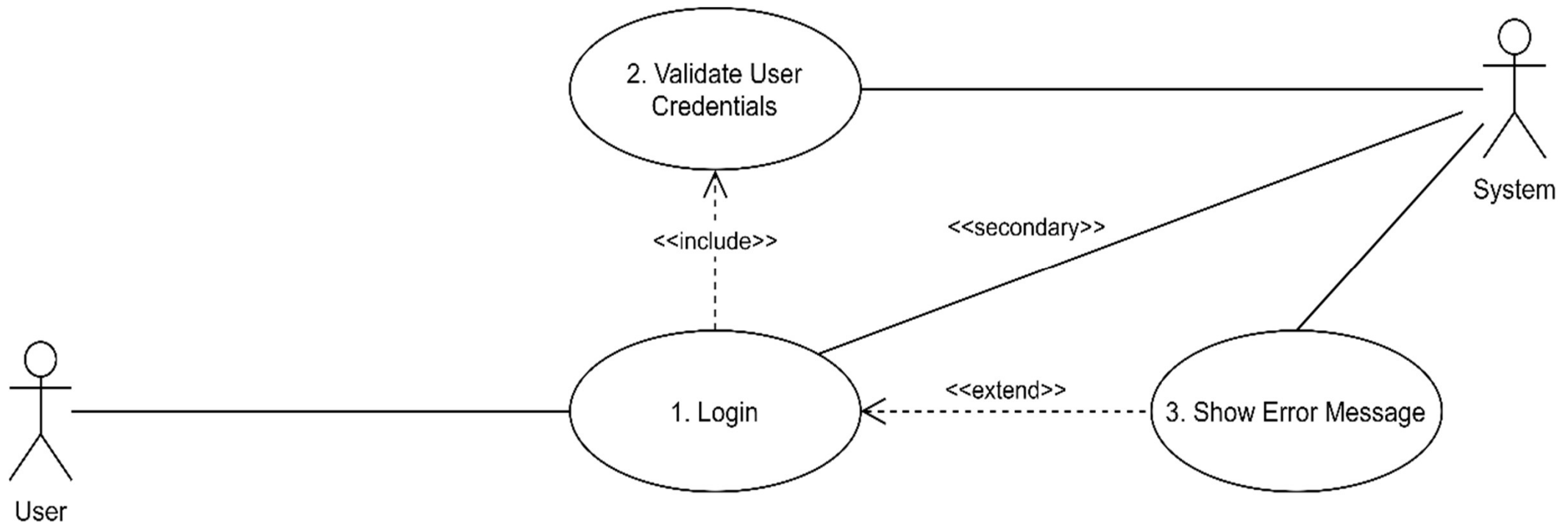
Use-case Diagram

Use-case Diagram

- Use-case diagrams model the **behavior of a system** and help to capture the **requirements of the system**
- Use-case diagrams describe the **high-level functions and scope** of a system **but not how the system operates internally**
- These diagrams also identify **the interactions between the system and its actors**
- You can model a complex system with a **single** use-case diagram, or create **many** use-case diagrams to model the components of the system

Use Case Diagram Notation

- Use case “Login”



Use Case Description Table

Use Case ID	1
Name	Login
Description	Verify user account information and validate identification
Primary Actors	User
Secondary Actors	System
Pre-conditions	None
Post-conditions	User is logged in
Main flow of events	1. System asks for username and password 2. User inputs data 3. System validates user credentials (Use Case ID: 2) 4. System prints authentication results
Exceptional Flow of events	2.1 Username is empty 2.1.1 Show error message (Use Case ID: 3) 2.2 Password is empty 2.2.1 Show error message (Use Case ID: 3) 3.1 Username and password cannot be validated 3.1.1 Show error message (Use Case ID: 3)



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Class Diagram

Class Diagram

- Class diagrams are fundamental to the object modeling process and model the **static structure of a system**
- Depending on the complexity of a system, you can use a **single class diagram** to model an entire system, or you can use **several class diagrams** to model the components of a system
- Class diagrams are the **blueprints** of your system or subsystem
- They are used to **model the objects** that make up the system, to display the **relationships between the objects**, and to describe **what those objects do** and the services that they provide

Class Diagram Relations Notation

Aggregation



Inheritance



Composition



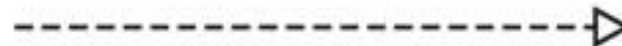
Association



Directed
Association

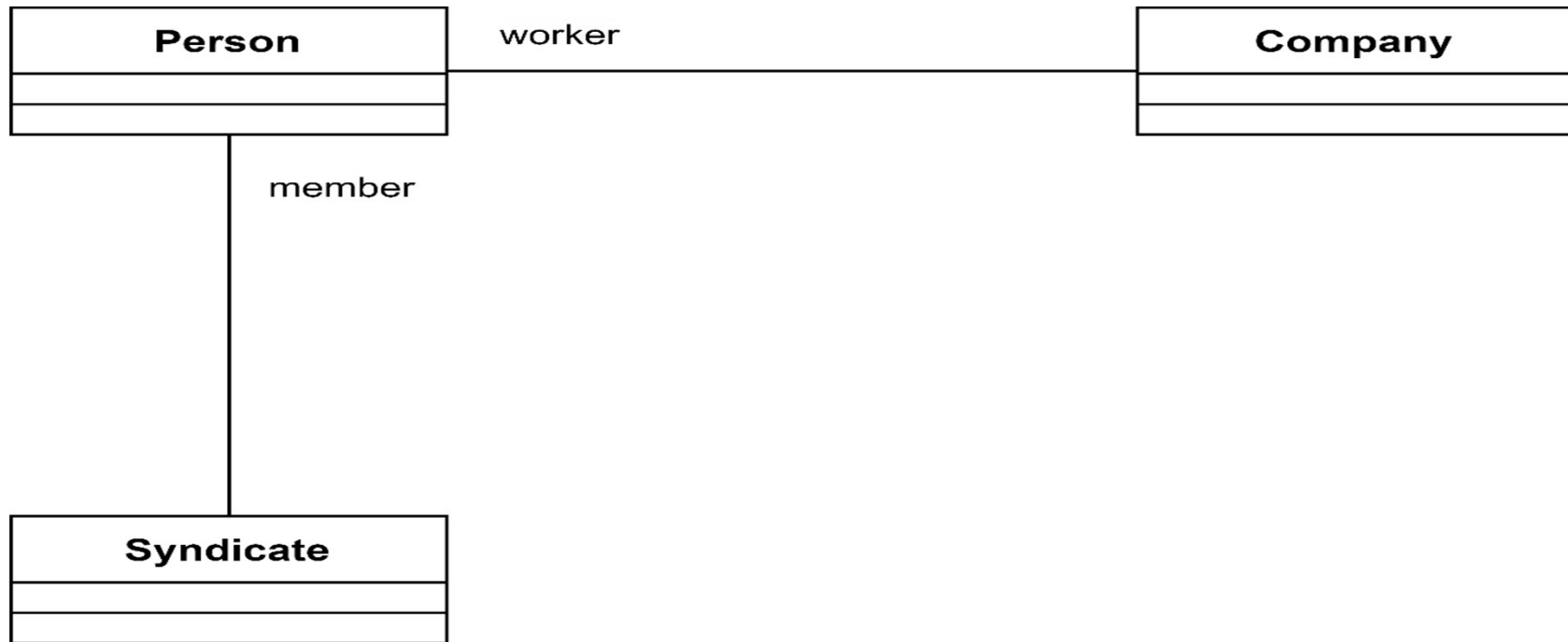


Interface Type
Implementation

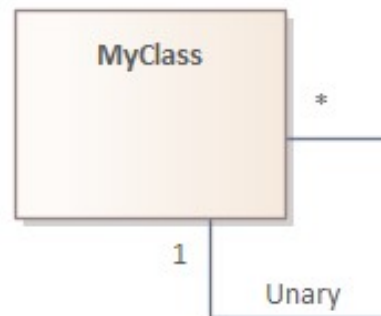


Class Diagram Relations Notes

Note1: You can use names as roles between classes



Note2: Association could be **Unary** if it is with the same class



Related topics

■ Cardinality:

- 1: Exactly one
- [Number]: Exactly this number
- 0..1: one or none
- 0..* (*): any number
- 1..*: at least one

■ Access level:

- - : private
- +: public
- ~ or #: protected

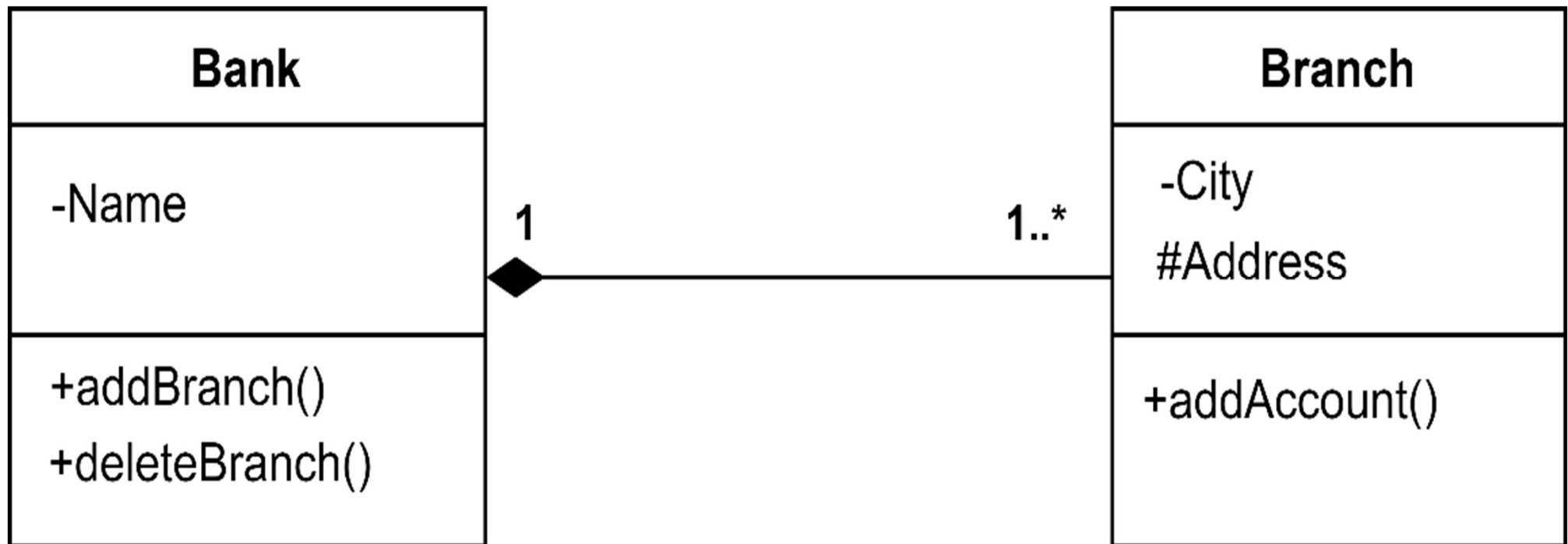
The access level

	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World
public	+	+	+	+	+
protected	+	+	+	+	
<i>no modifier</i>	+	+	+		
private	+				

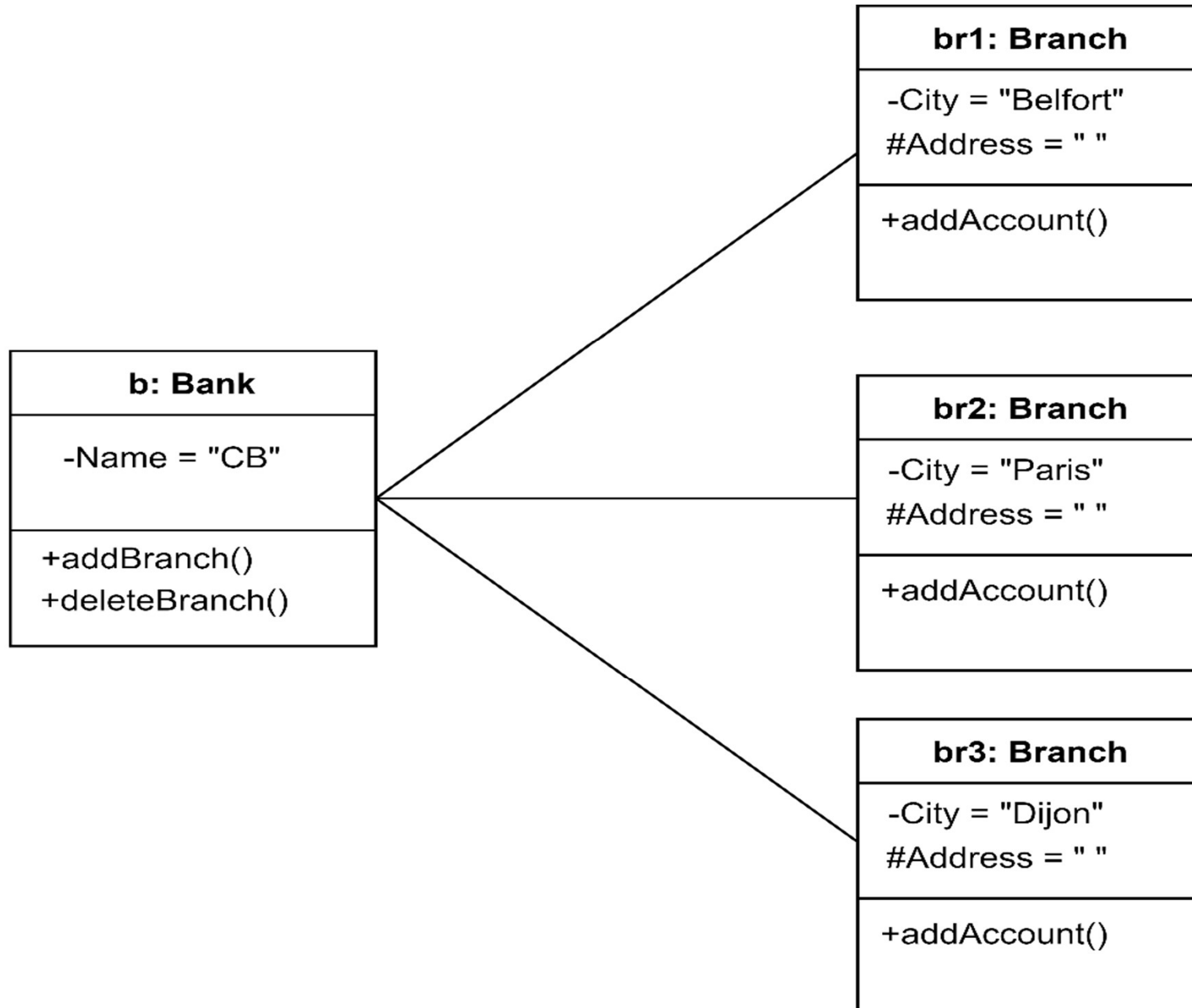
+ : accessible

blank : not accessible

Class Diagram Example



Object Diagram Example



Steps of preparing a class diagram

- Steps of preparing a class diagram:
 1. Define the classes
 2. Define any relations of Inheritance
 3. Add data members and operations
 4. Define the association relations between classes
 5. Refine the association relations into other types of relations, e.g., composition
 6. Improve the relations with cardinalities, directions, and roles



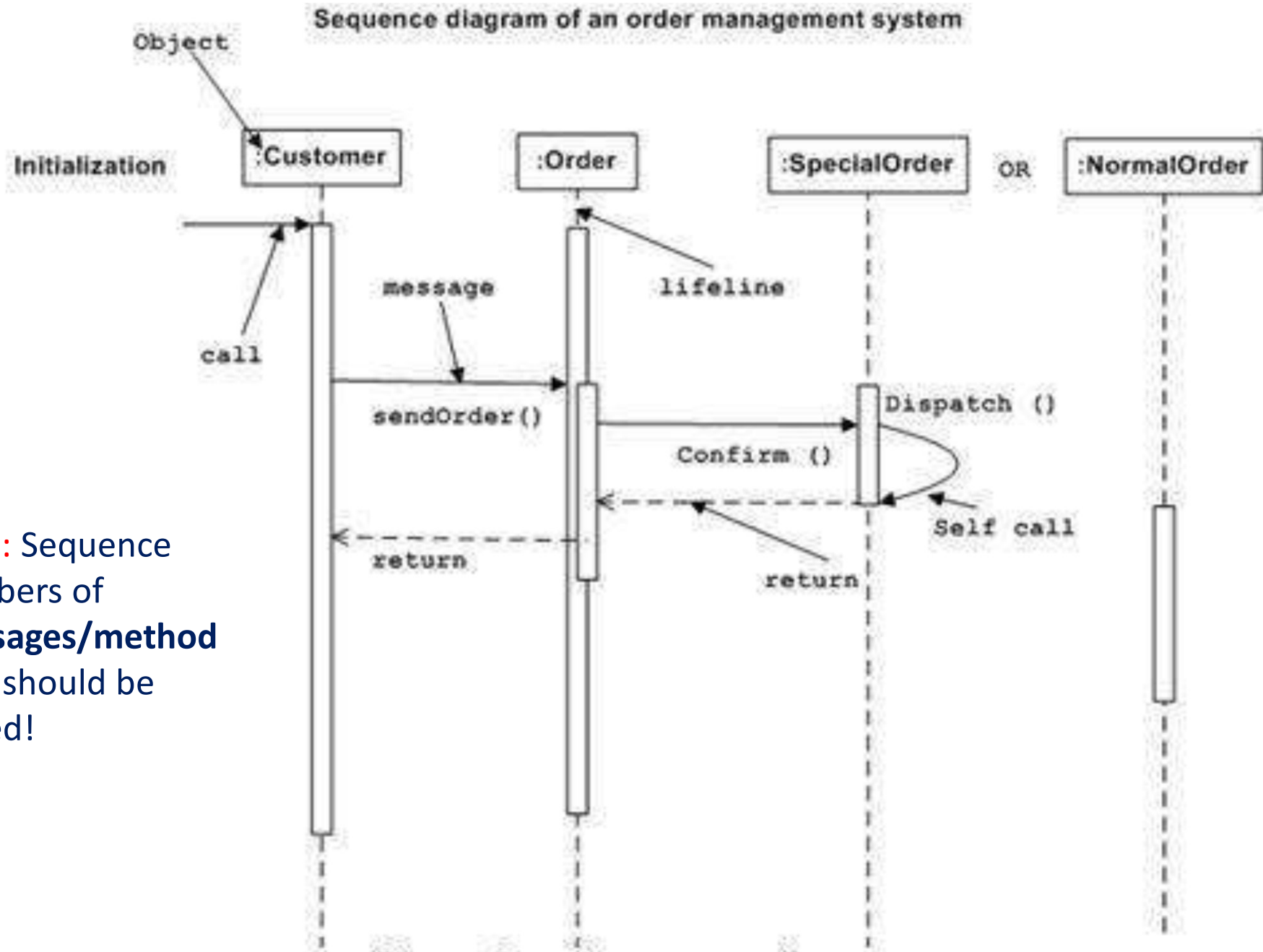
UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Sequence Diagram

Sequence Diagram

- Sequence Diagrams are interaction diagrams that **detail how operations are carried out**. They illustrate **the sequence of messages between objects in an interaction**
- Sequence Diagrams are **time focus** and they **show the order of the interaction visually** by using the vertical axis of the diagram to represent time, what messages are sent and when
- It specifically focus on **lifelines**, or the processes and objects that live simultaneously, and the messages exchanged between them to perform a function before the lifeline ends

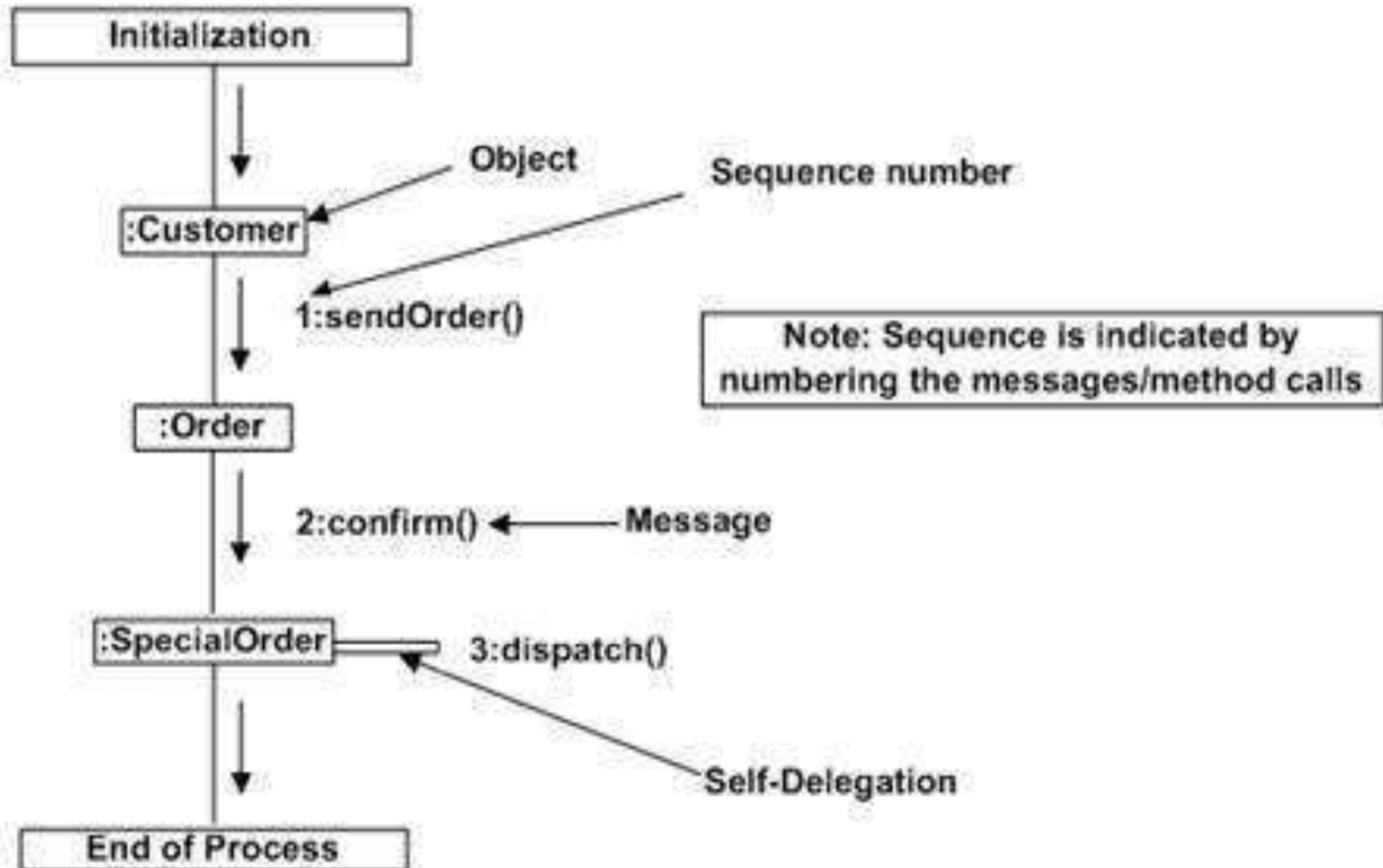
Sequence Diagram Example



Note: Sequence numbers of **messages/method calls** should be added!

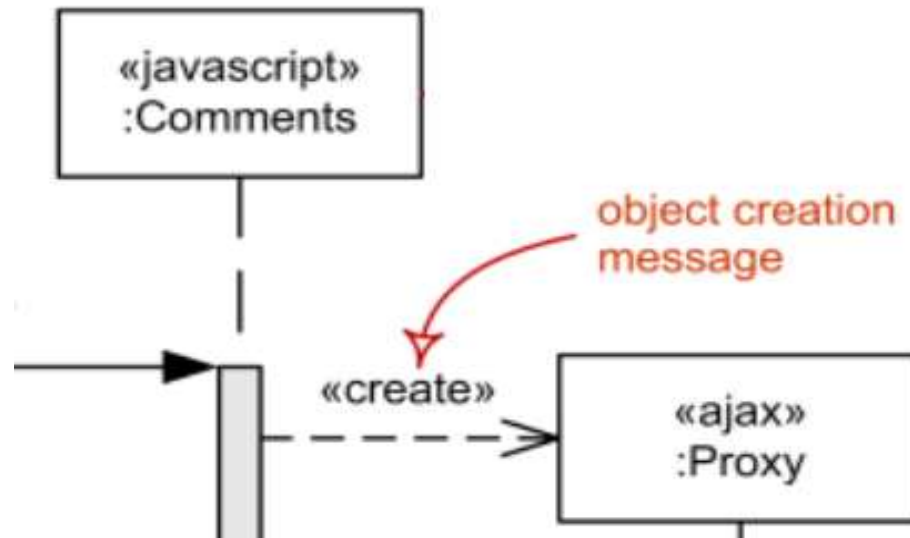
Collaboration Diagram Example

Collaboration diagram of an order management system

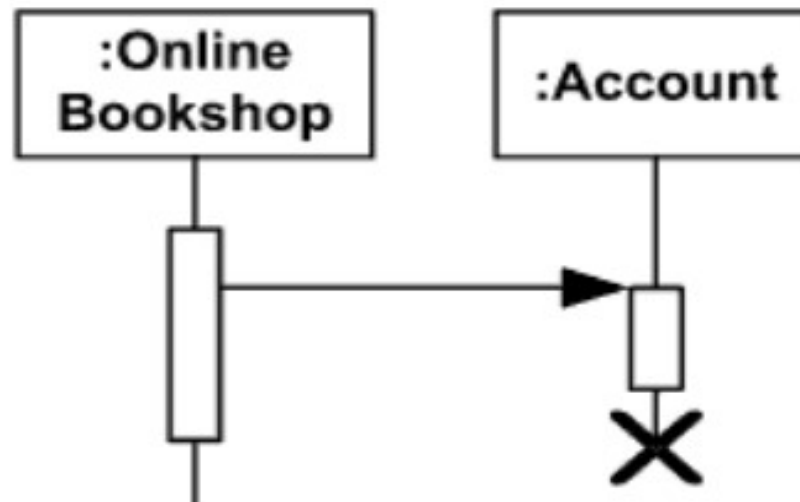


Sequence Diagram: Other notations

- Creation:

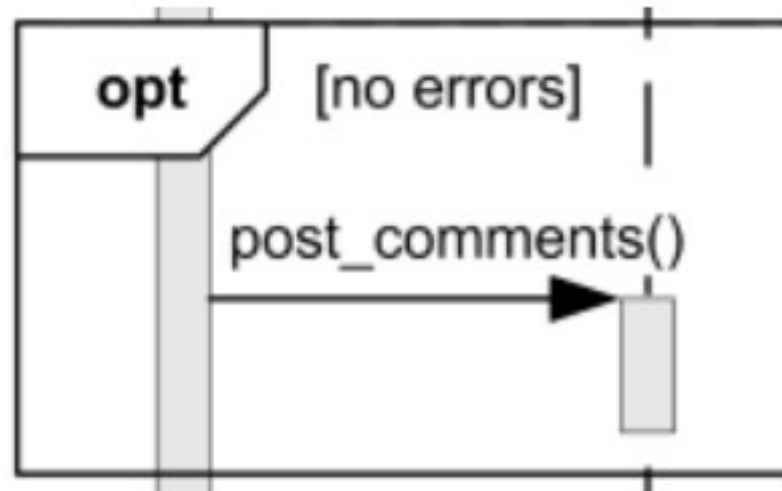


- Destruction:

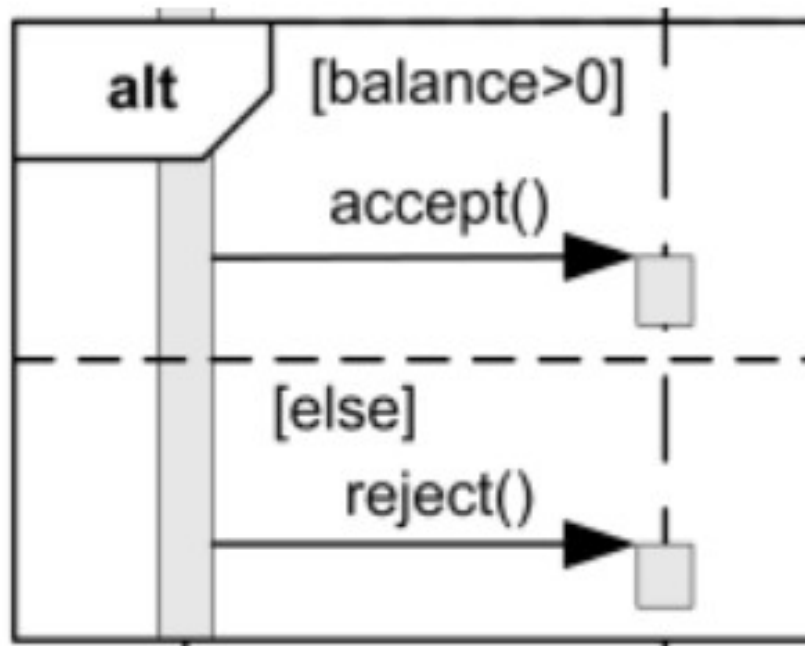


Sequence Diagram: Other notations

- Option:

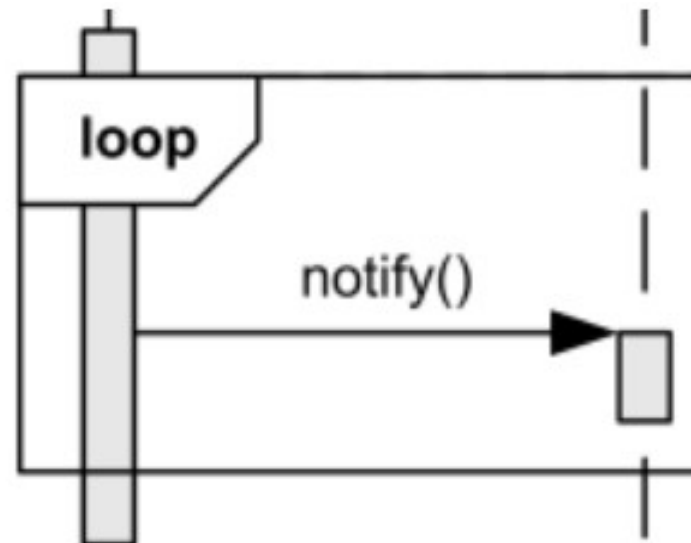


- Alternatives:

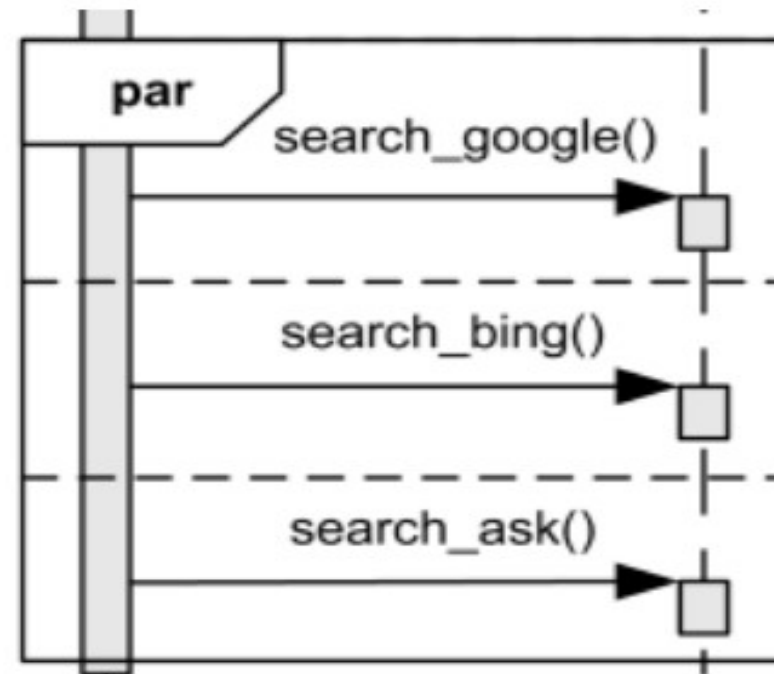


Sequence Diagram: Other notations

- Loop:



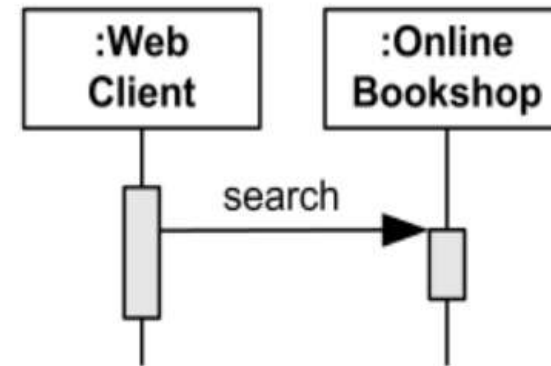
- Parallel:



Sequence Diagram: Other notations

- Synchronous Call (**filled arrow head**):

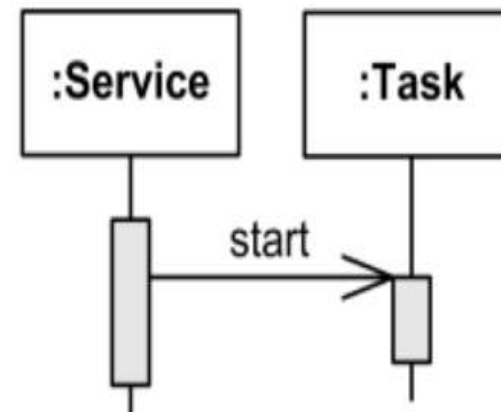
It typically represents an operation call - send a message and suspend execution while waiting for response.



Web Client searches Online Bookshop and waits for results.

- Asynchronous call (**open arrow head**):

It sends a message and proceeds immediately without waiting for the return value.



Service starts Task and proceeds in parallel without waiting.



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

State Machine Diagram

State Machine Diagram

- A state machine diagram models the **behavior of a single object**, specifying the sequence of events that an object goes through during its lifetime in response to the events
- A state machine diagram is a **graphical representation of the states of an object**, the events that cause a transition from one state to another, and the actions that result from a change in state
- State machines can also contain **nested states or sub state-machine diagrams** that represent different hierarchical state levels. You can use nested states to examine complex state changes in objects

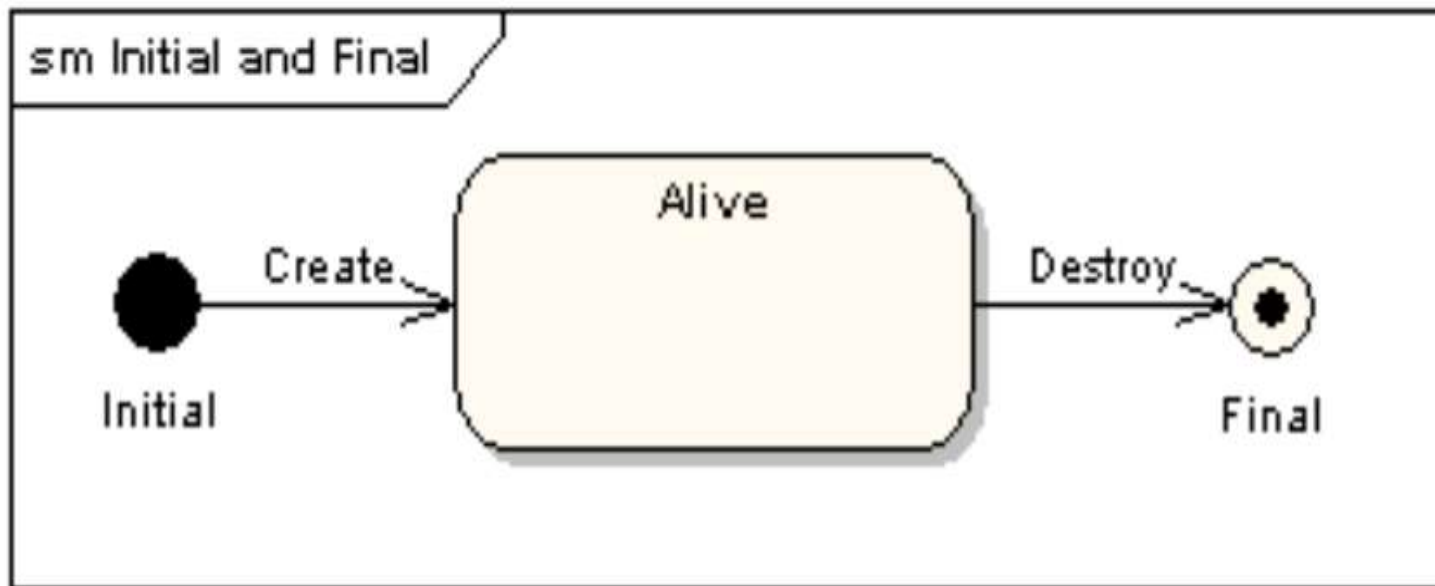
Introduction Examples

- Door:
 - Open
 - Closed
 - Locked

- Computer:
 - Turned-off,
 - Turned-on,
 - Hibernated,
 - Sleeping,
 - Hanging,
 - Broken,
 - Etc.

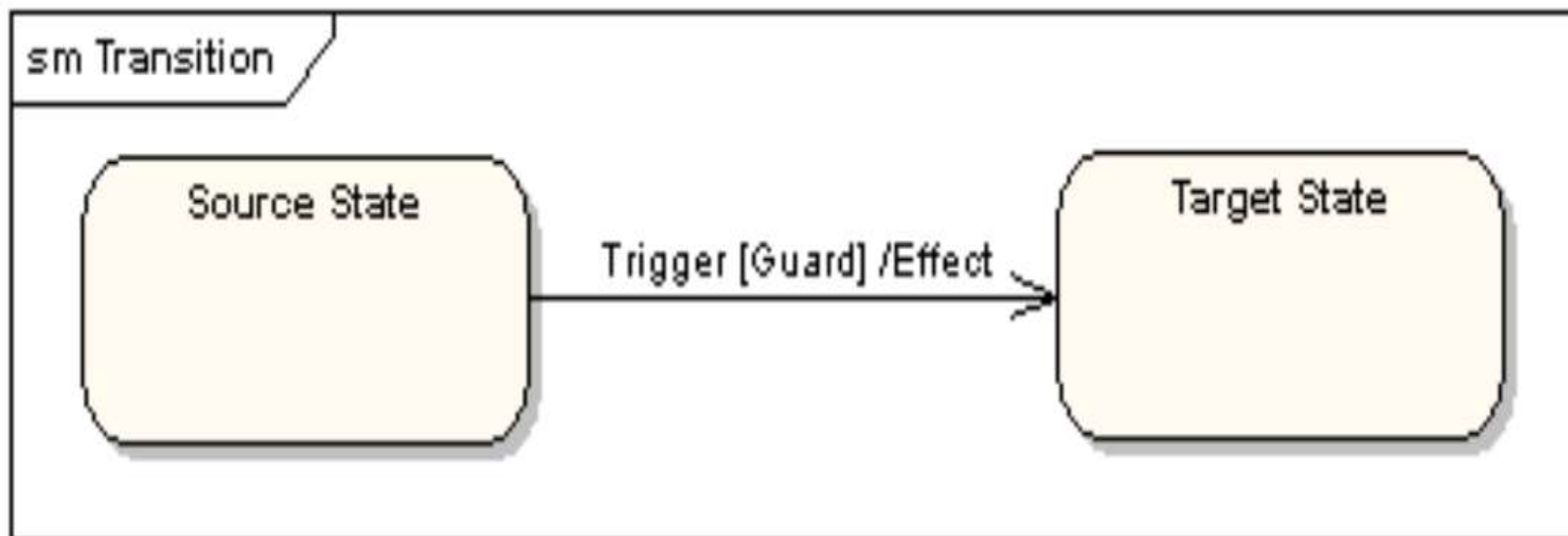
State Machine Diagram: Notation

- Initial and Final States



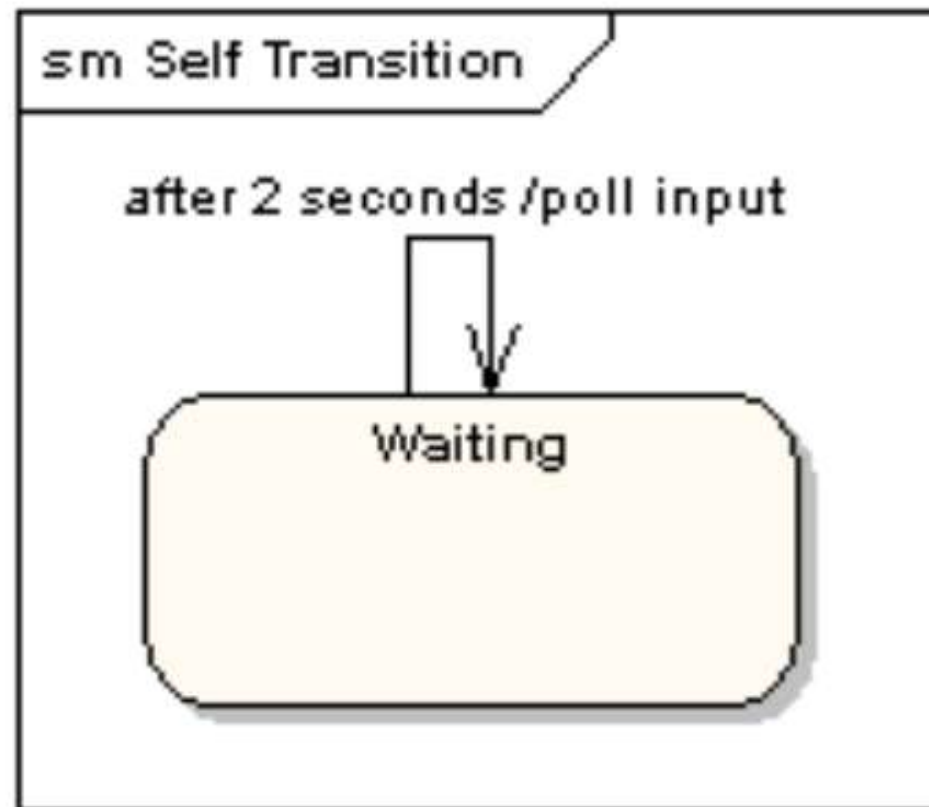
State Machine Diagram: Notation

- **Transitions** from one state to the next are denoted by lines with arrowheads. A transition may have
 - **Trigger:** the cause of the transition, which could be a signal, an event, a change in some condition, or the passage of time
 - **Guard:** a condition which must be true in order for the trigger to cause the transition
 - **Effect:** an action which will be invoked directly on the object that owns the state machine as a result of the transition

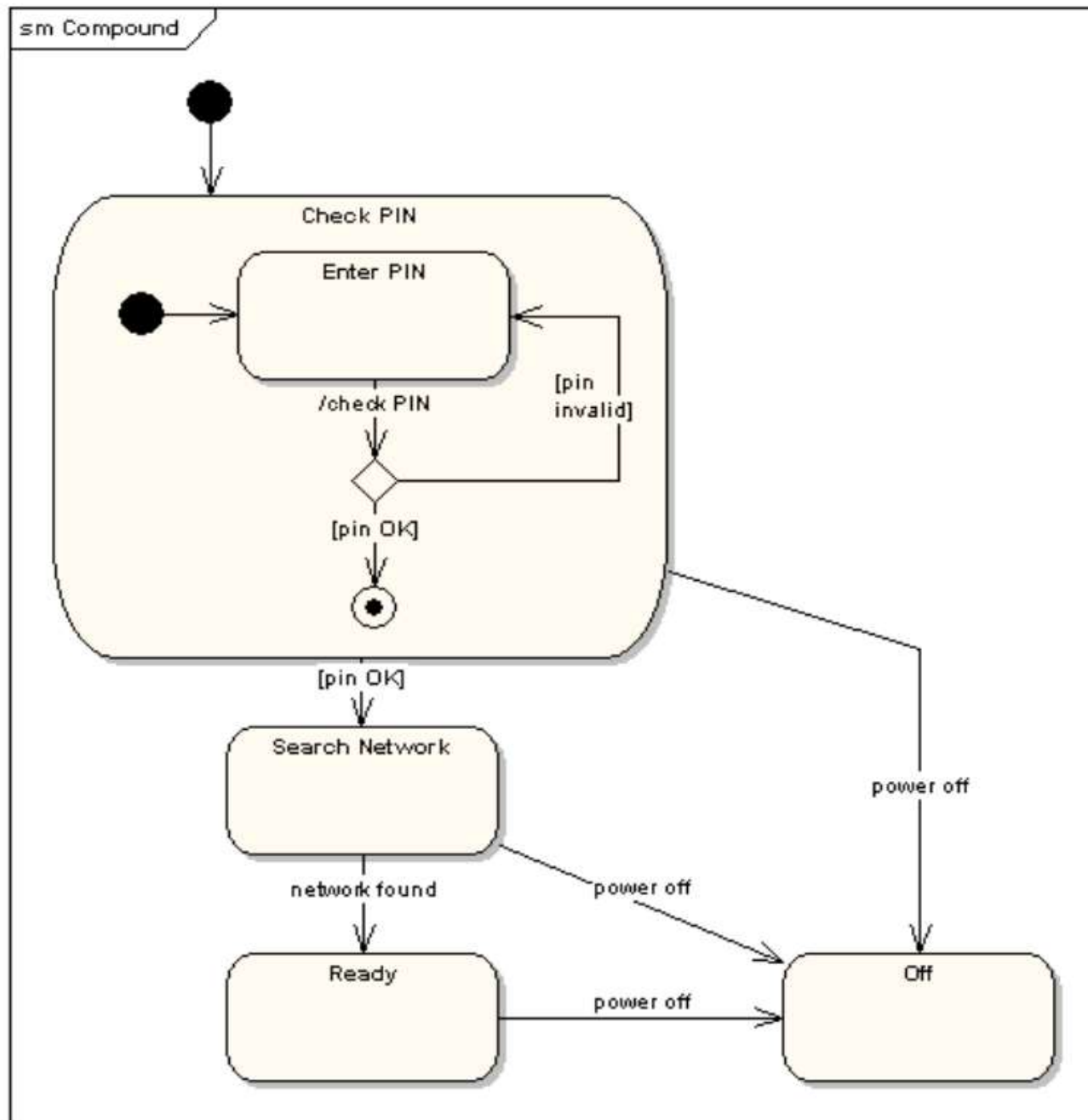


State Machine Diagram: Notation

- Self-Transitions

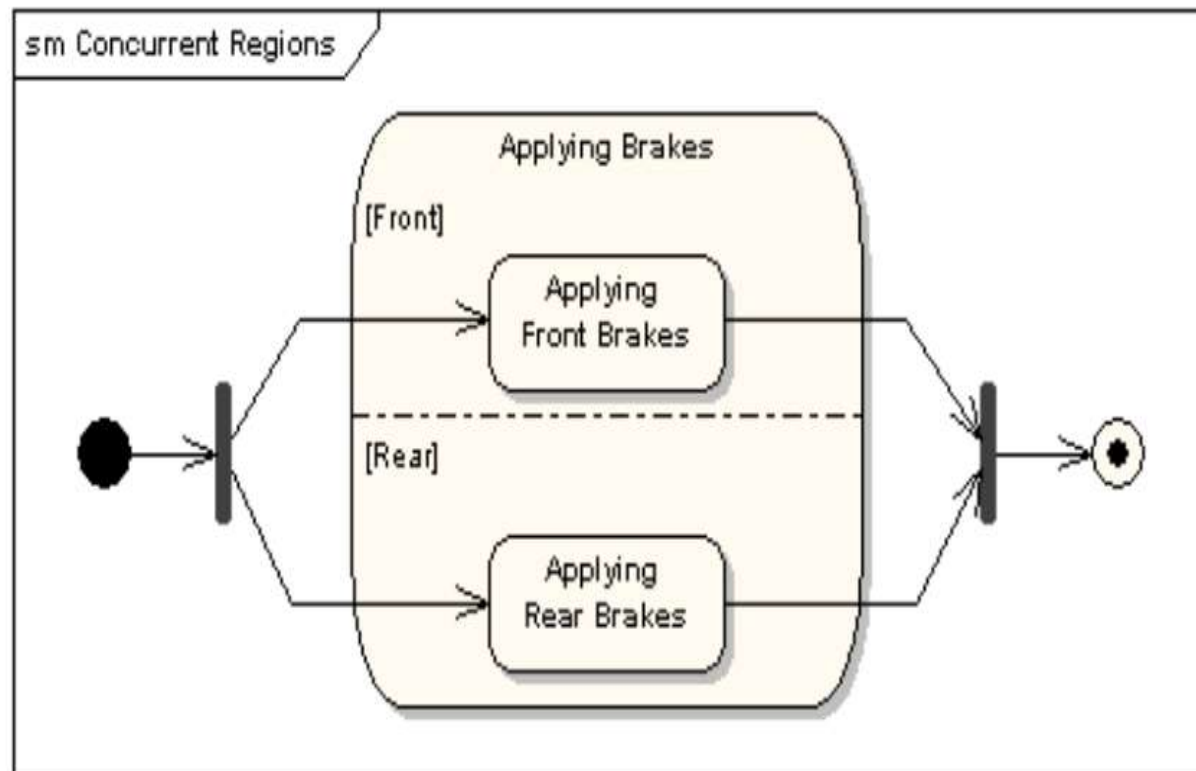


State Machine Diagram: Compound States

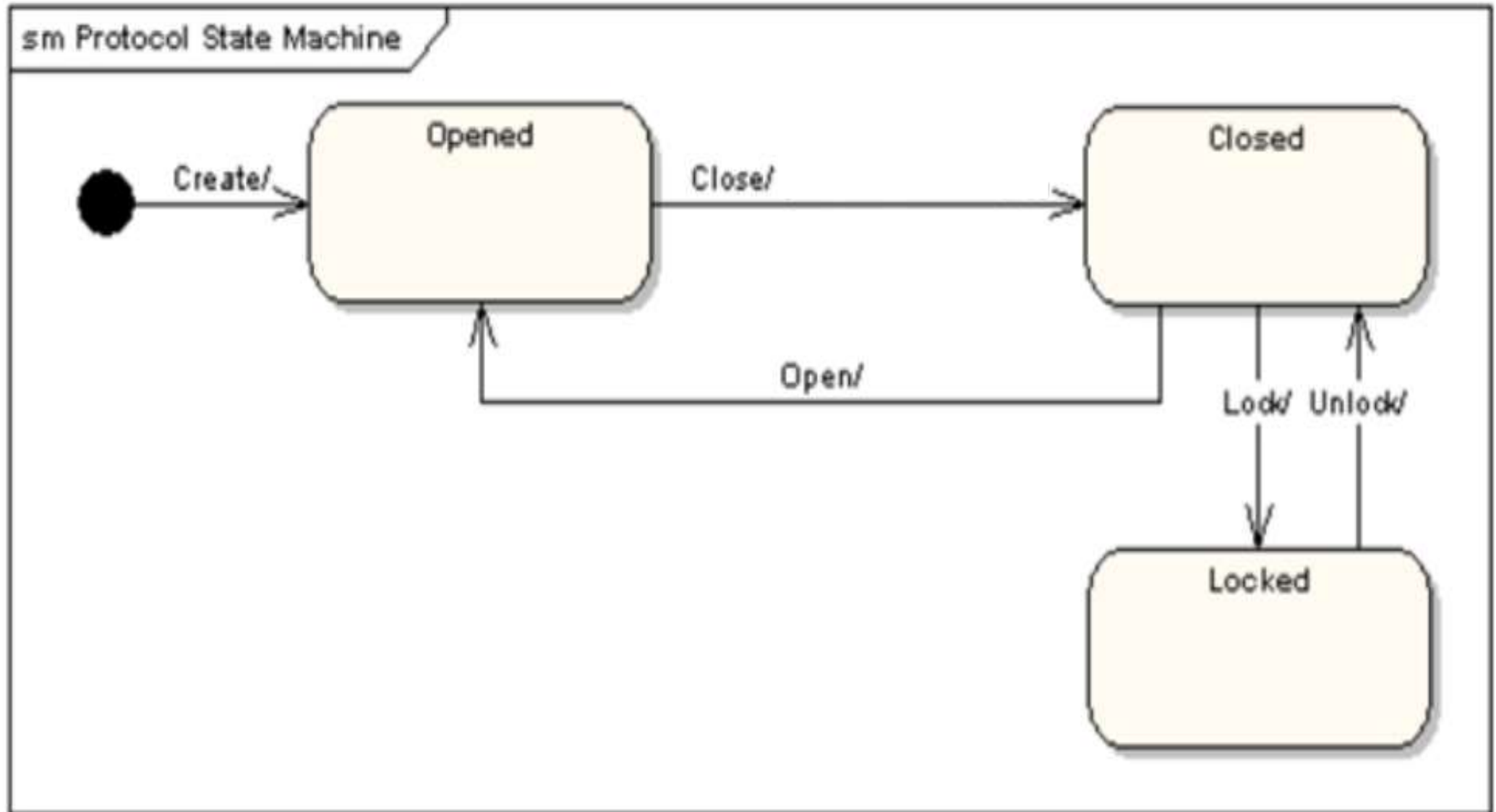


State Machine Diagram: Notation

- **Concurrent Regions:** A state may be divided into regions containing sub-states that exist and execute concurrently. The following example shows that within the state "Applying Brakes", the front and rear brakes will be operating simultaneously and independently. Notice the use of **fork** and **join** pseudo-states. These symbols are used to synchronize the concurrent threads



State machine of a door



Note: For more background details check Deterministic Finite Automata (DFA)

UML References

- Reference for more details:
 - <https://www.uml-diagrams.org/sequence-diagrams.html>
 - Official website where you can download the full UML standard (specification) with the latest version:
<https://www.omg.org/spec/UML>

Note: Current UML version is 2.5 (slightly different naming convention)

References and Acknowledgments

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- University of Alberta Computing Science, Faculty of Science
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- “Object-Oriented Patterns,” P. Coad; Comm. of the ACM, 9/92
- “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

End of Lecture